# A Private Blockchain for Jointly Signed Negotiated Rates Between Providers and Payers

**Abhinav Srivastava**[1]

[1]**Independent Researcher; abhinavmirdev@gmail.com**

## Abstract

*Providers and payers each publish their negotiated rates. A 2026 comparison of 450 hospitals and two payers found about one in ten paired rates match exactly and about half fall within 10 percent. Each file is published by one party without the other's confirmation. CORAL is a permissioned ledger on which a provider and a payer jointly sign each negotiated rate with its plan, code, modifier, setting, formula, dates, and contract version. Providers sign chargemaster versions, so percentage rates recompute from signed inputs. Published machine-readable files cite the record identifier, so a line that departs from the signed rate is detected by anyone who holds the file. We implement CORAL on Hyperledger Besu and evaluate it on public provider and payer files, reporting disagreement before and after signing, the party responsible for each remaining difference, and cost per contract.*

## Working notes (remove before submission)

Everything built for this paper is recorded here. Dates are absolute. Numbers come from runs in the repository (`https://github.com/AbhinavMir/coral`) unless a source is named. Say provider, not hospital, except in a regulation name or a cited study.

## Problem

- Providers publish negotiated rates under 45 CFR 180 (the Hospital Price Transparency rule). Payers publish in-network rates under the Transparency in Coverage rule (85 FR 72158). Each file is produced by one party. No file carries the other party's signature.
- Trilliant Health, 30 April 2026: 450 hospitals, Aetna and UnitedHealthcare, 16 codes, data from August 2025. Exact match 9.0% inpatient, 10.8% GI, 11.7% orthopedic. Within 10%: 50.8%, 41.7%, 47.4%. More than 50% apart: 17.9% for inpatient and GI.
- The comparison cannot say which file is correct. Neither file links a line to a contract version, an effective date, or the other party.
- Rates are formulas: a fixed amount, a percent of billed charges, or a percent of a reference fee schedule. In the one real provider file profiled (UCHealth Yampa Valley Medical Center, file dated 2025-11-01, CMS format 3.0.0): 633,324 rows, 454,793 with a dollar amount, 228,164 (36%) percent of total billed charges, 30 payer/plan pairs, one type 2 NPI in the header (1790787307). A percent-of-charges line changes value when the chargemaster changes. The payer does not sign the chargemaster.

## Terms

- Negotiated rate: the price a provider and a payer agreed for one code. Contractual. The CMS file calls it the payer-specific negotiated charge.
- Chargemaster: the provider's list price for every item. Set by the provider alone, not negotiated. The CMS file calls it the gross charge. A percent-of-charges rate is a percent of this number, so the contract fixes the percent and the provider fixes the base.
- Setting: inpatient (admitted) or outpatient (not admitted). The same CPT or HCPCS code is often paid differently in each. Admissions as a whole are priced under MS-DRG codes, which exist only for inpatient stays. The CMS file has a setting column with the values inpatient, outpatient, and both; the record adds professional.
- Fields that identify one rate: payer, plan, billing NPI, code, modifier, setting, and date. Two rates equal on all of these are the same rate.

- In the CMS file format but not in the record: drug unit of measure (per mg, per mL, per unit) and billing class (facility or professional). Gap: a per-mg rate and a per-vial rate for one HCPCS code are different rates and the record cannot tell them apart. To add.
- In contracts but in neither the file nor the record: place of service beyond the setting split, revenue code, bundling rules, multiple-procedure reductions, outlier and stop-loss terms, carve-outs, per-diem tiers, annual escalators. The rule requires a rate per code, not the rules for combining codes on a claim. Out of scope: the paper checks the published line.

**Prior art**

- No paper or product found on jointly signed negotiated rates or a ledger that holds rate records (six searches, 27 October 2026).
- Claim-status networks on a ledger exist: Change Healthcare on Hyperledger Fabric (2018), Avaneer Health, Synaptic Health Alliance (provider directory), MediLedger 340B working group (2021). None holds the rate with both signatures.
- Transparency logs (Certificate Transparency, Trillian, SCITT) are append-only under one operator. A ledger with several validators is justified only when the parties distrust each other and accept no single operator. Cite the three-condition test from the blockchain-health position paper. Equivocation detection: PeerReview (SOSP 2007), SUNDR fork consistency. arXiv 2308.04781: an append on a chain is not proof the counterparty received it.
- FHIR Provenance, W3C PROV, C2PA, HL7 AI Transparency on FHIR cover record provenance, not contract terms.

**What is new**

- One record, one hash, two signatures. A provider and a payer sign the same rate before either publishes it. No existing network does this.
- A record id that both published files and an X12 claim can carry. Verification of a file is a lookup, not a comparison of two files.
- A mismatch between the two public files resolves to "provider file departs" or "payer file departs". Today it resolves to nothing.
- Percent-of-charges rates become checkable: the provider signs the chargemaster version, the rate names it, the amount recomputes from signed inputs.
- Key validity judged at commit time, so a rotated key does not invalidate old records.
- The ledger is not the novelty. It is the neutral place to timestamp the record when CMS, providers, and payers accept no one of the others as operator.

**Keys: identity roots**

- Provider contracting entity: the tax identification number (TIN). Contracts are signed at the TIN. A TIN covers one or more type 2 NPIs. CMS binds NPIs to organizations through NPPES and PECOS and authorizes staff through the Identity & Access Management System.
- Payer entity: a payer id. Candidates: the HIOS issuer id (used in Transparency in Coverage files), the NAIC company code, or the X12 payer id. The prototype uses an opaque string.
- Registrar: CMS. CMS receives both files and proofs both kinds of organization. CMS binds an entity id to a root public key once, after out-of-band proofing. CMS never signs a rate.

- Entity ids in the prototype: `TIN:84-0398876` (UCHealth Yampa Valley Medical Center, TIN from the file name, NPI 1790787307 from the file header), `PAYER:UHC`, `PAYER:AETNA`. Entity names are stored as written in the file so a published file's payer column matches the registry. Any string up to 32 bytes works; longer ids are hashed and the file must carry the full text.

**Keys: hierarchy**

- Root key: one per entity, held offline. It signs delegations and revocations only. It never signs a rate; `commitRate` checks the signer against delegations and a root key has none.
- Signing key: one or more per entity, delegated by the root for a window [notBefore, notAfter). These live in the contract-management system that produces the rate sheet. Rotation = delegate a new key, revoke the old one from now.
- Delegation and revocation records are on the ledger. A verifier answers "was key K authorized for entity E at time T" from the ledger alone (`authorizedAt`).

**Keys: time**

- Validity is checked at the block timestamp of the commit, set by IBFT consensus, not by the signer. A key revoked at T cannot commit at T+1, and a compromised key cannot backdate a commit to before its revocation.
- Records committed before a revocation stay valid (`test_revocation_blocks_future_commits`).
- The ledger's clock is the contract's clock. Effective dates inside the rate (start, end) are contract terms signed by both parties and are separate from the commit time.

**Keys: signature format**

- EIP-712 typed data. A signing tool shows the fields to the person who signs. The domain separator binds chain id and contract address, so a signature cannot be replayed on another deployment (`test_signature_bound_to_deployment`
- Both parties sign the same struct hash. The record id is that hash. Anyone can relay the transaction; the relayer has no authority. In the prototype CMS runs the relayer and is the registrar.
- Two signatures, one transaction. The first signature is held off-ledger in the app's pending store (`build/pending.json`: proposer, proposer signature, rate, time) until the second party countersigns. A proposal never countersigned leaves no ledger trace. An on-ledger proposal record, which would show who refused to sign, is deferred.

**Keys: failure cases**

- Signing key compromised: root revokes from the time of compromise. Records after that time by that key are invalid; records before it stand. Which to contest is a contract question.
- Root key lost or compromised: CMS replaces the root (`replaceRoot`) after re-proofing. Existing delegations stay until the new root revokes them.
- Registrar key compromised: a governance action by the validators. Out of scope; note it.
- Validator node keys are separate from record signing keys. The four validators stand for CMS, a provider association, a payer association, and an auditor. No validator can sign a rate.
- X.509 or DirectTrust certificates could stand in for the root key. The ledger still needs delegation and revocation records to answer "valid at time T" without an OCSP responder. The contract only needs the root's public key.

**Design**

- Record key: payer id, plan id, provider TIN, NPI, code, modifier, setting. Record value: formula (fixed, percent of charges, percent of reference), value (cents or basis points), start, end, contract version, reference version.
- Record types on the ledger: Entity (registrar writes), Delegation and Revocation (root signs), Rate (both sign), ChargemasterVersion (provider signs a SHA-256 of the file, a version label, and an effective date). Amendment, witness checkpoint, and on-ledger proposal are deferred.
- Verifier: recomputes the amount from signed inputs only. Fixed: the value. Percent of charges: value times the list price for the code and setting in the chargemaster version the record names. Percent of reference: value times the reference amount in the named schedule version (implemented in contract and verifier, not exercised on real data).
- Verdict per published line: match, amount differs, no record (never signed), expired (signed but the window does not cover the date), ambiguous (more than one live record for the key), needs chargemaster (percent rate and no list price available).
- Provider MRF lines and payer TiC lines carry the record id. X12 837 and 835 could carry it in a REF segment; claims are out of scope.
- Scope: one check. Does a published line match a jointly signed record that covers its date.

**Build: ledger**

- Contract `contracts/RateRegistry.sol`, Solidity 0.8.24, 7,220 bytes of bytecode, 35 ABI entries. EVM Shanghai (PUSH0). Event `RateCommitted` trimmed to five fields to avoid stack-too-deep.
- Network: Hyperledger Besu 26.10.2, IBFT 2.0, four validators in Docker (`besu/docker-compose.yml`), 2 s block period, zero base fee, chain id 2741. Fixed IPs because Besu rejects hostnames in enode URLs.
- Client `coral/chain.py`: web3 7 with the POA extra-data middleware and local signing; `coral/typed.py`: EIP-712 types and signing helpers; `coral/compile.py`: solc build cache. In-memory EVM (eth-tester) for tests.
- Reads come from events. Besu caps `eth_getLogs` at 5000 blocks per call; `Chain.logs` scans from the deploy block (stored in `build/deployment.json`) in 4000-block ranges. Found when the app returned 500 after block 5000.
- `RateIndex`: loads every record once and indexes by key, for bulk verification. Per-line RPC lookups were quadratic and timed out on the first evaluation run.
- Tests (`tests/test_registry.py`, 11, all pass): commit and read, duplicate, unauthorized key, wrong-side key, revocation, delegation window, root cannot sign, domain binding, root replacement, verify fixed, verify percent of charges.

**Build: files and matching**

- `coral/mrf.py`: `provider_header` reads the two-row header (provider name, date, version, type 2 NPI); `provider_csv` yields one line per negotiated row of a CMS v2/v3 tall CSV, with payer names mapped to registered payer ids and plan names used as plan ids; `payer_json` streams a Transparency in Coverage in-network file with ijson and resolves provider references to NPIs (written, not yet run on a real file); `gross_charges` indexes list price by code and setting from the gross charge column.
- `coral/verify.py`: `Line` (one published line, normalized), `verify_line` (one verdict), `expected_amount`.
- `coral/match.py`: pairs provider lines with payer lines on the record key and reproduces the Trilliant measure (exact, within 10%, over 50%); with ledger verdicts it attributes each remaining difference to the file that departs (provider file departs, payer file departs, both match, unsigned). Written; the payer side has not run on real data.

**Build: app**

- FastAPI and static HTML (`app/`). Signing keys are server-held (`build/keys.json`, clear text, prototype only); a deployment replaces one function with the entity's own signing service.
- Provider/payer page: Negotiated rates (the whole ledger, readable by anyone; one search box; each record opens to effective dates, contract, both signers, commit time), Pending (countersign, reject, withdraw), Propose (form; loads one of ten examples), Chargemaster (provider uploads a file; the server signs its hash and keeps the file under `build/chargemasters/<sha256>`), Keys (rotate). The "sign as" choice exists only because the server holds every key.
- CMS page: Registry (entity, kind, name, NPI, root key, signing keys and their state) and Verify a published file (provider CSV or payer JSON; the NPI comes from the file header; payer names in the file are matched to the registry; per-line verdicts and counts).
- Percent-of-charges records show the computed amount; hover shows percent, list price, and chargemaster version. The ledger holds only the hash; the server reads the list price from the stored file.
- Removed on 2026-10-29: export, coverage counts, claim-line check, block list. Claims are out of scope; the rest were views, not actions.
- Examples (`data/examples.json`, ten records, `scripts/seed_examples.py` signs them): codes, plans, settings, list prices, and amounts copied from the UCHealth file (CT head, ED level 3, chest X-ray, metabolic panel, colonoscopy, knee arthroplasty, MS-DRG 470 and 871, enoxaparin; fee schedule, case rate, percent of charges). Contract and chargemaster version labels are marked as examples because the file does not carry them. The chargemaster version signed is the SHA-256 of the file itself, which holds the list prices.
- `scripts/bootstrap.py`: deploys the contract, registers the three entities, delegates one signing key each. `scripts/evaluate_provider.py`: the evaluation below.

**Results (run 2026-10-27, `results/evaluate_provider.json`)**

- Input: UCHealth Yampa Valley Medical Center file, payer "United Healthcare", plan "Broad Networks". First 200 distinct (code, modifier, setting) lines with a dollar amount.
- Parse: 0.8 s for the 141 MB file to the first 200 lines.
- Commit: 200 records, one per block, 399 s wall, 1.99 s per record. Gas 258,923 mean per record. Transaction input 740 bytes mean. Sequential because the client waited for each receipt; the block gas limit allows many per block.
- Verify, clean copy: 200 of 200 match. Index load 2.2 s, verification 2.2 s for 200 lines.
- Verify, tampered copy (20 of 200 amounts altered by $\pm 10\%$, +50%, or $-50\%$): 180 match, 20 amount differs. No false positives, no misses.
- Key rotation on the ledger: new key delegated, old key revoked from now; a commit with the old key reverts with `SignerNotAuthorized`; `authorizedAt(old, commitTime)` true and `authorizedAt(old, now)` false.
- Example records: the three Aetna percent-of-charges records compute to $67.39, $310.65, and $1,290.10 from the signed file, equal to the file's negotiated dollar column.
- The payer signature in these runs is a prototype key, not UnitedHealthcare's agreement. The runs show mechanism and cost, not that the file is correct.

**Open items**

- Payer side on real data: one Transparency in Coverage in-network file for the same provider (UHC publishes an index at transparency-in-coverage.uhc.com), run `payer_json` and `match.py`.
- The Trilliant pairing (450 hospitals): provider files are found through each provider's `cms-hpt.txt`. Of ten providers tried on 2026-10-27, four served it (NYP, UCHealth, OHSU, Houston Methodist); the rest returned HTML or a block page.
- Add drug unit of measure and billing class to the record key.
- Batched commits, N per block; measure records per second.
- On-ledger proposal record. Amendments that reference the record they replace. Both deferred.
- Paper body: 10 pages, AMIA format, Vancouver references.

**Venue**

- AMIA Annual Symposium, Paper - Regular: 10 pages, abstract 125-150 words, one column, Letter, Vancouver references. 2027 symposium 6-10 November, San Diego. Call for papers not yet out; past deadlines mid March.
- Alternatives: IEEE ICBC 2027 (deadline 14 January 2027), Blockchain in Healthcare Today (10,000 words, USD 1,285 APC, PubMed Central), Journal of Medical Systems (IF 5.7).